

Information Diffusion in Social Networks

CAPSTONE PROJECT PHASE-2

Phase – II Report

Submitted by

1. 21BCE11078 Braj Kishor Sharma
2. 21BCE11159 Arijit Modak
3. 21BCE10076 Devashish Dhaulakhandi
4. 21BCE11072 Aswin P Kumar
5. 21BCE11650 Akshant Ujjwal

in partial fulfillment of the requirements of the degree of

Bachelor of Engineering and Technology



VIT[®]
BHOPAL
www.vitbhopal.ac.in

VIT Bhopal University
Bhopal
Madhya Pradesh

Jan 9, 2025



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Bonafide Certificate

Certified that this project report titled “**Information Diffusion on Social Networks**” is the bonafide work of **21BCE11078 Braj Kishor Sharma, 21BCE11159 Arijit Modak, 21BCE10076 Devashish Dhaulakhandi, 21BCE11072 Aswin P Kumar, 21BCE11650 Akshant Ujjwal** who carried out the project work under my supervision.

This project report (DSN4092-Capstone Project Phase-II) is submitted for the Project Viva-Voce examination held on **9th January 2025**.

Supervisor

Comments & Signature (Reviewer 1)

Comments & Signature (Reviewer 2)

INDEX

Chapter Number	Topic	Page No.
1	Introduction	1
1.1	Motivation	2
1.2	Objective	3
2	Literature Review	4-5
3	Frontend, Backend and System Requirement	6-19
4	Methodology	20-24
5	Conclusion	25-26
6	Reference	27
7	Team Members -Biodata with Picture	28-30

CHAPTER 1: INTRODUCTION

The spread of information, behaviors, and innovations across networks is a fundamental phenomenon that shapes many aspects of modern society, from marketing and social influence to public health and policy-making. In an increasingly connected world, online social networks play a pivotal role in accelerating the diffusion of ideas, with both positive and negative consequences. Understanding the dynamics of information propagation within these networks is essential for developing strategies to enhance the reach of beneficial innovations, mitigate harmful misinformation, or optimize marketing campaigns.

This research focuses on simulating and analyzing the diffusion process of information using a simplified Susceptible-Infected (SI) model, which provides a framework for modeling how information spreads through a population of interconnected individuals. By drawing from key concepts in graph theory and network analysis, we explore how different network structures influence the speed and extent of diffusion. The study is inspired by influence maximization techniques, which aim to identify the optimal nodes in a network for initiating diffusion to maximize its reach.

To visualize and analyze this process, we employ two powerful tools: Neo4j, a graph database that facilitates the exploration and manipulation of complex networks, and NetworkX, a Python library designed for the creation, manipulation, and study of the structure and dynamics of complex networks. These tools enable us to model and simulate diffusion processes, capturing the inherent complexities of real-world social networks.

Our approach goes beyond theoretical models by incorporating 3D visualizations to better capture the dynamic and spatial aspects of information flow in networks. These visualizations allow for a deeper understanding of how information propagates, highlighting key factors such as network topology, node centrality, and the role of influential individuals in accelerating diffusion. Through this work, we aim to provide both theoretical insights and practical strategies for optimizing information dissemination in various domains, ranging from marketing to public health interventions.

1.1 Motivation

The rapid proliferation of information through social media platforms has revolutionized communication, but it has also given rise to significant challenges, most notably the spread of misinformation. False or misleading information can spread quickly across networks, amplifying its impact and creating widespread societal, political, and economic consequences. Understanding how information, both true and false, propagates within these networks is crucial for developing strategies to mitigate harm and control the flow of information.

At the heart of this issue lies the need to identify key points in the network where interventions can have the greatest impact, whether in curbing misinformation or accelerating the spread of beneficial innovations. By modeling the dynamics of information diffusion, we can uncover patterns and identify influential individuals or nodes within the network that serve as critical leverage points for managing the flow of ideas and behaviors.

In parallel, businesses and marketers face similar challenges in optimizing the spread of marketing campaigns, products, and services. Companies are increasingly turning to data-driven methods to understand how information diffuses within social networks, seeking to target influencers who can help amplify their message and maximize campaign outreach. Identifying these key influencers is vital for maximizing engagement, improving brand awareness, and boosting sales.

Moreover, enhanced visualization tools like Neo4j Bloom and NetworkX provide an invaluable aid in this process, enabling businesses to visualize complex social networks and understand the flow of information across them. These visualizations offer a deeper understanding of network structures, highlighting the most influential individuals and optimizing strategies for both intervention and marketing campaigns. Through these tools, businesses can refine their targeting strategies, while policymakers and health professionals can implement more effective public health interventions to curb misinformation or promote healthier behaviors.

Thus, the motivation for this research stems from the dual need to combat misinformation and enhance marketing and influence strategies. By simulating and analyzing information diffusion in dynamic networks, we aim to provide actionable insights that can be used across various domains to optimize information flow and intervention strategies.

1.1 Objective

The primary objective of this research is to develop a comprehensive understanding of how information diffuses across social networks, with a focus on both theoretical exploration and practical application. By simulating the spread of information through a simplified Susceptible-Infected (SI) model, we aim to explore the dynamics of diffusion in social networks and identify strategies that can optimize the reach and effectiveness of information dissemination. Specifically, the research seeks to achieve the following objectives:

1. **Simulate and Model Information Diffusion:** Construct a dynamic, agent-based model based on the SI framework to simulate how information spreads through various types of networks. This includes the implementation of the model in real-world network topologies, capturing the impact of structural features such as node centrality, network connectivity, and community structures on the diffusion process.
2. **Investigate Factors Influencing Diffusion Efficiency:** Explore the role of different network characteristics—such as degree distribution, clustering coefficient, and path length—in shaping the speed and extent of information spread. This objective seeks to uncover how the structure of the network impacts the effectiveness of diffusion, with a particular focus on identifying factors that affect the flow of information.
3. **Identify and Optimize Key Intervention Points:** Utilize influence maximization algorithms to identify critical nodes within the network that serve as key drivers of information spread. This objective aims to identify influential individuals or clusters within a social network who can be targeted to accelerate the diffusion of positive information or slow the spread of harmful misinformation.
4. **Enhance Visualization and Interpretation of Diffusion Processes:** To aid in the intuitive understanding of the information diffusion process, we will integrate React Force Graph, an open-source visualization tool, into our platform. This interactive, 3D visualization tool will allow users to explore how information propagates through the network in real time. The graph will display the connections and interactions among nodes, making it easier to track the spread of information, identify influential nodes, and simulate different intervention strategies.

By addressing these objectives, this research aims to contribute to a better understanding of information diffusion in social networks and strategies for both mitigating negative outcomes (such as the spread of misinformation) and enhancing positive outcomes (such as the effective spread of public health messages or marketing campaigns).

CHAPTER 2: EXISTING WORK / LITERATURE REVIEW

Information diffusion and influence maximization (IM) have become important research areas due to their applications in social networks, marketing, and public health initiatives. The main challenge is to identify influential nodes, or seed nodes, that can initiate the widest spread of information or behaviors across a network. Researchers have proposed numerous models and algorithms to balance accuracy and computational efficiency in solving this problem.

The SI Model: A Simplified Approach to Information Diffusion

The SI model is based on a system of ordinary differential equations that tells about the Susceptible individuals and infected individuals, with their rate of spreading the infection.

SI Model consists of two main parts :

1. **Susceptible (S):** Individuals who have not yet received the information but are potential recipients.
2. **Infected (I):** Individuals who have acquired the information and can transmit it to others.

The SI (Susceptible-Infected) model operates on the key assumption that individuals who acquire information remain in the "infected" state indefinitely. Once they receive the information, they continue to share it without transitioning out of this state. This simplification allows the model to focus entirely on the process of spreading information, without considering real-world factors such as forgetting, losing interest, or ceasing to share the information. While this assumption streamlines the analysis of growth dynamics, it does not account for behaviors like disengagement or social influences that may cause individuals to stop spreading information over time.

The dynamics of the SI model are governed by the following differential equation:

$$dI/dt = \beta \times (S \times I) / N$$

Here:

- β represents the rate at which information is transmitted.
- S denotes the count of individuals susceptible to receiving the information at time t .
- I indicates the number of individuals actively spreading the information at time t .
- N is the total population size, calculated as $N=S+I$.

This equation models the rate at which susceptible individuals become infected through interactions with infected individuals. Since:

$$dS/dt = - dI/dt$$

The decrease in susceptibles is equal to the increase in infected individuals.

Rationale for Using the SI Model

The SI model is the most basic compartmental model in epidemiology and tends to be used simply as a framework to understand the rudimentary dynamics of disease spread. Once infected, they remain infectious indefinitely, simplifying the equations and the analysis, making it ideal for certain scenarios.

- **Early-Stage Epidemics:** In the SI model, this can be a reasonable approximation during initial outbreak periods when there are limited recovery rates; alternatively, it may be appropriate not considering at all the recovering dynamics only in the early-time spread.
- **Educational and Theoretical Contexts:** In the theoretical study of the dynamics of epidemics, the SI model allows mathematical treatment to be handled by a few of the main concepts: herd immunity thresholds, network effects, and population heterogeneity.

Comparison with SIR Model

The SIR model is widely used in simulations of information, disease, and behavior diffusions within networks. Originating from epidemiology, the SIR model categorizes nodes into three states: Susceptible (S), Infected (I), and Recovered (R).

If the period of infection is relatively short in comparison with the lifetime of a person and assuming the infection does not kill the victims, birth and death factors can be ignored. Thus, the deterministic SIR model is given by the system of ODEs:

- $dS/dt = - \beta \times (S \times I) / N$
- $dI/dt = \beta \times (S \times I) / N - \gamma \times I$
- $dR/dt = \gamma \times I$

Transmission Probability (β): The likelihood that an infected node will transmit the information to a susceptible neighbor.

Recovery Rate (γ): The rate at which infected nodes transition to the recovered state.

CHAPTER 3 : FRONT END, BACKEND AND SYSTEM REQUIREMENT

3.1 Front End

The front end of our Information Diffusion Simulation project has been carefully designed to offer an interactive and intuitive experience for users, allowing them to explore and understand the dynamics of information diffusion within a network. The core functionality of the front end revolves around the 3D visualization of a network, where users can observe the real-time spread of information across nodes. Below is a detailed overview of the key components and features available in the frontend:

1. Interactive 3D Visualization:

We utilize the react-force-graph library to render a 3D force-directed graph, where nodes represent individuals and edges signify their connections. This visualization offers an intuitive understanding of information flow within the network.

2. Dataset Integration:

Currently, our application supports interactive 3D visualizations for three distinct social-interaction datasets namely InVS13, InVS15 & Lyon School, allowing users to explore, compare and study different network structures and Visualize information diffusion patterns.

3. Simulation Controls:

We have provided a slider component that enables users to visualize the step-by-step progression of information flow throughout the network. This feature is complemented by real-time statistics displaying the percentage of informed and uninformed populations, providing immediate insights into the simulation's dynamics.

4. Node Interaction:

Users can interact directly with the graph to obtain detailed information about each node, including node identifiers and their current states. This functionality enhances user engagement and understanding of individual node behaviors within the network.

5. User Controls:

Users can initiate or stop the simulation via control buttons, which are conditionally enabled based on the simulation's state, ensuring a seamless user experience.

These features collectively enhance the application's capability to simulate and visualize information diffusion, providing users with deeper insights and a more engaging experience.

Technical Workflow of Front End:

The front end of the Information Diffusion Simulation follows a user-friendly workflow that guides the user through selecting datasets, running simulations, and visualizing results in real-time. This workflow is designed to ensure a seamless and interactive experience, enabling users to both observe and manipulate the information diffusion process in a straightforward way. Here is an overview of the process:

1. **Dataset Selection:**

Upon entering the application, users are first prompted to select a dataset on which they wish to run and visualize the simulation. This ensures that the simulation is tailored to their specific data set.

2. **Simulation Page:**

After choosing a dataset, users are directed to the simulation page, where they can view the initial state of the network represented in 3D. This provides a clear, dynamic view of the graph before the information diffusion process begins.

3. **Start Simulation:**

To initiate the simulation, users simply click the Start Simulation button. This triggers the diffusion process, and the 3D graph begins to evolve in real-time.

4. **Interactive Slider:**

A **slider** is displayed, allowing users to control the progress of the simulation. They can move the slider to navigate to specific steps or states within the diffusion process. This gives users the ability to pause and explore specific moments of the simulation.

5. **Statistics View:**

Users can also view real-time statistics, such as the percentage of informed vs. uninformed nodes in the network. This percentage is dynamically updated as the simulation progresses, providing insights into how information spreads through the network over time.

6. **Control Simulation:**

At any point during the simulation, users can interact with the slider to adjust the simulation step and observe how the network state evolves. If the user wants to halt the simulation, they can click the Stop Simulation button. This will freeze the simulation, allowing them to analyze the final state of the network and the diffusion process.

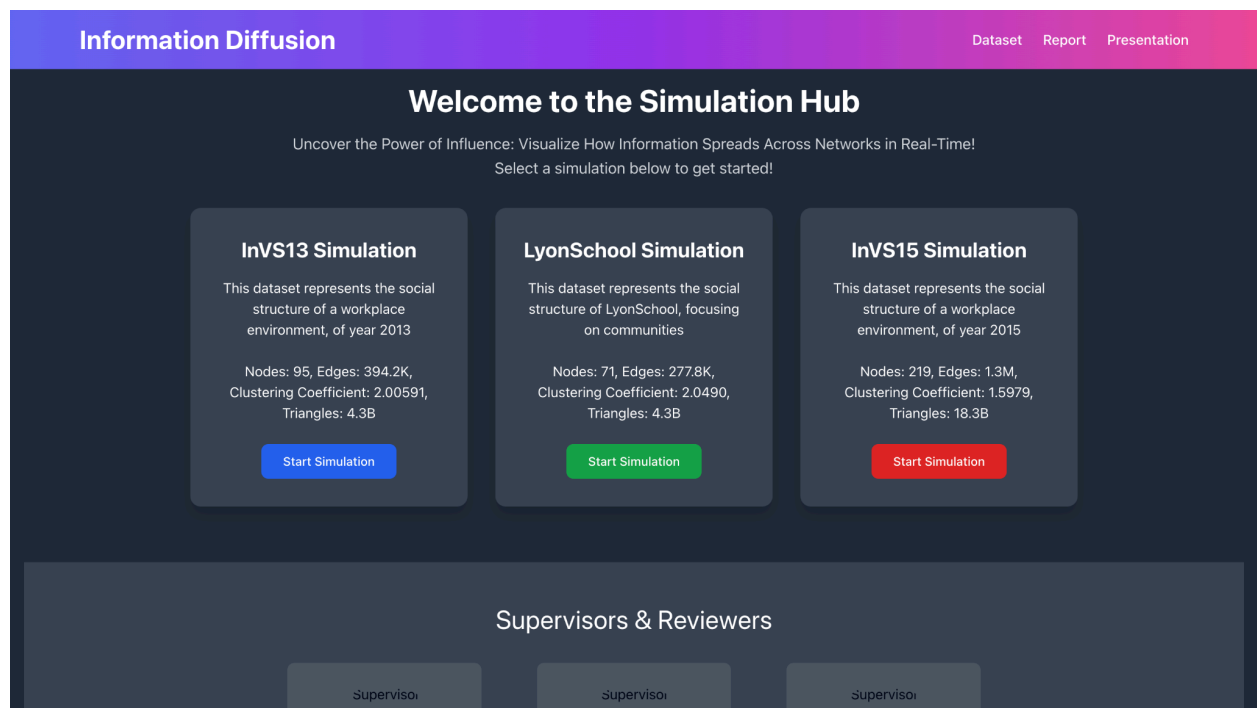


Figure 3.1.1: Home Page

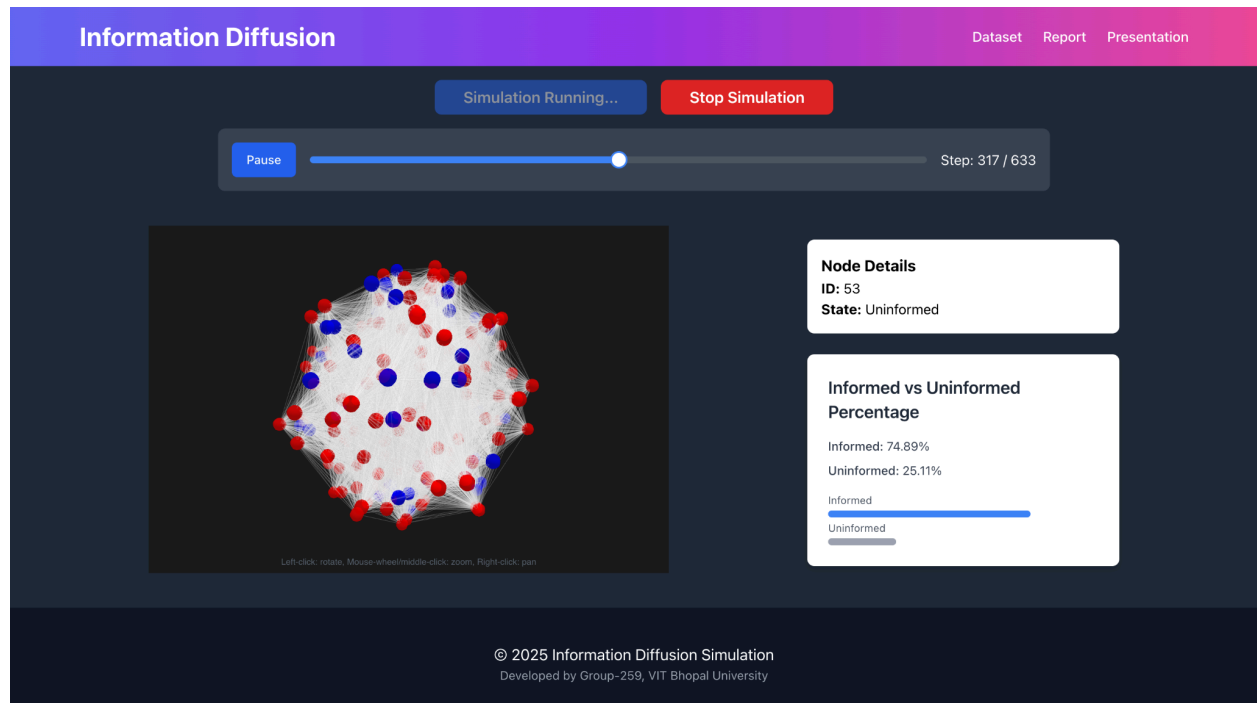


Figure 3.1.2: 3D-Simulation Page

3.2 Data Collection

The given project works with three co-presence data sets of different social network setup: The contexts in which these data were collected are: a workplace, with data collected in two different years (InVS13, InVS15) and a primary school (LyonSchool) to analyze them and model their graph structure. Each dataset provides persons and the relation among them—that is, nodes and edges—which give a view on the dynamics of each of these social networks.

1. [InVS13](#)

This dataset represents the social structure of a workplace environment, with entities (referred to as PID) assigned to different groups (or teams) within the organization. It includes:

- Nodes: 95
- Edges: 394.2K
- Clustering Coefficient: 2.00591
- Triangles: 4.3B

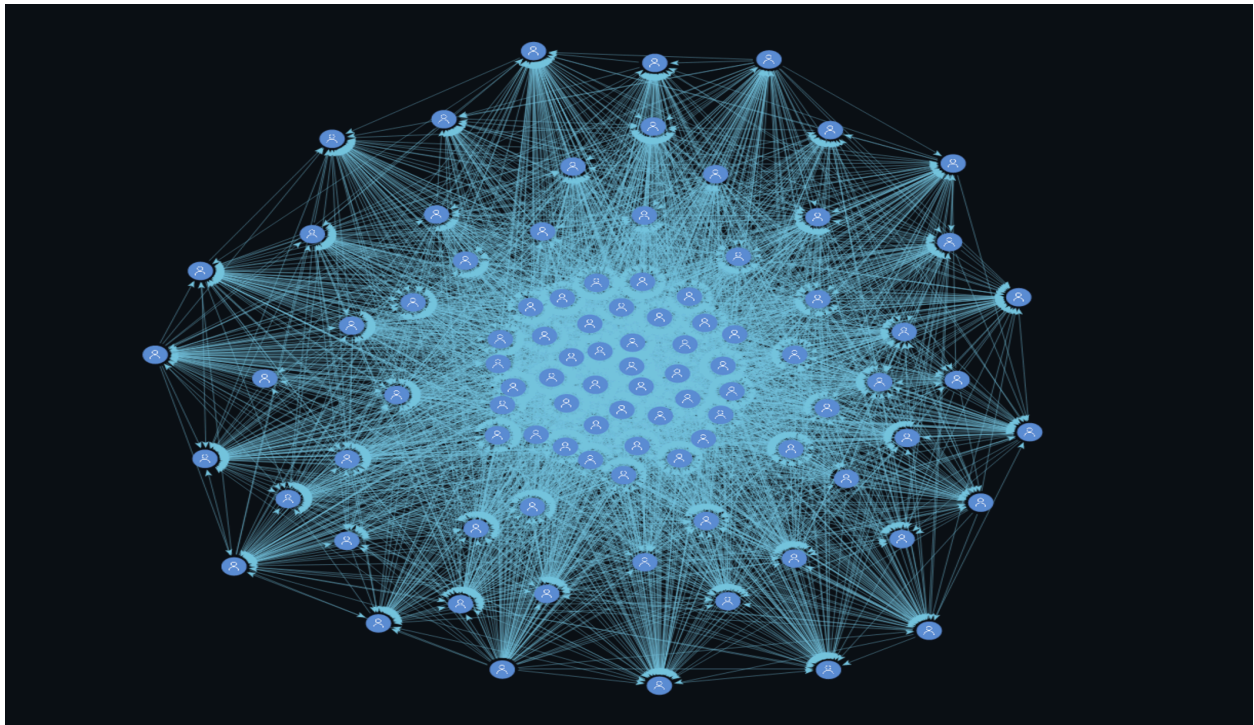


Figure 3.2.1: InVS13 Visualization

2. [InVS15](#)

This dataset represents the social structure of a workplace environment, with entities (referred to as PID) assigned to different groups (or teams) within the organization. It includes:

- Nodes: 219
- Edges: 1.3M
- Clustering Coefficient: 1.5979
- Triangles: 18.3B

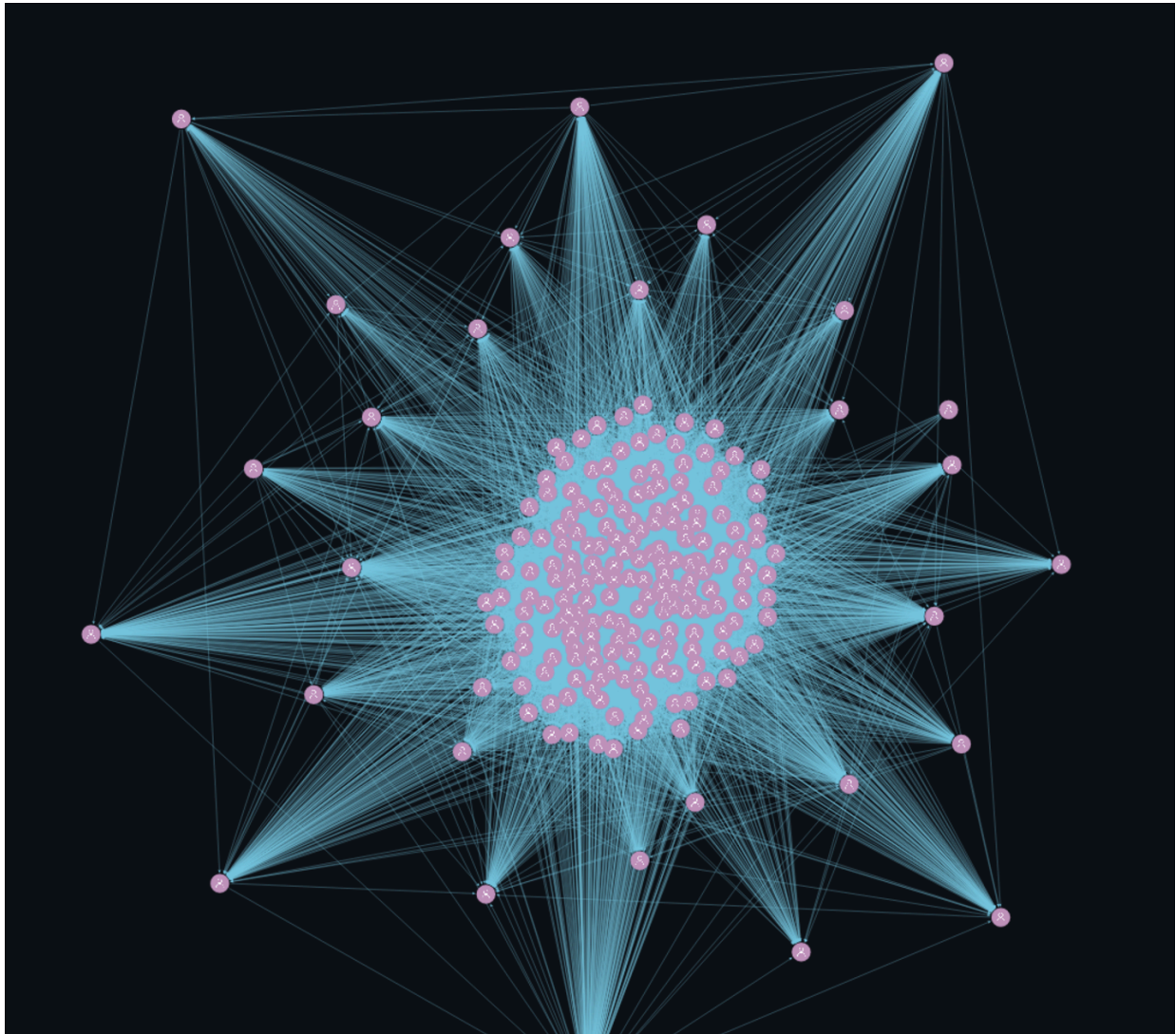


Figure 3.2.2: InVS15 Visualization

3. [LyonSchool](#)

This dataset represents the social structure of LyonSchool, focusing on user-defined communities. It includes:

- Nodes: 71
- Edges: 277.8K
- Clustering Coefficient: 2.0490
- Triangles: 4.3B

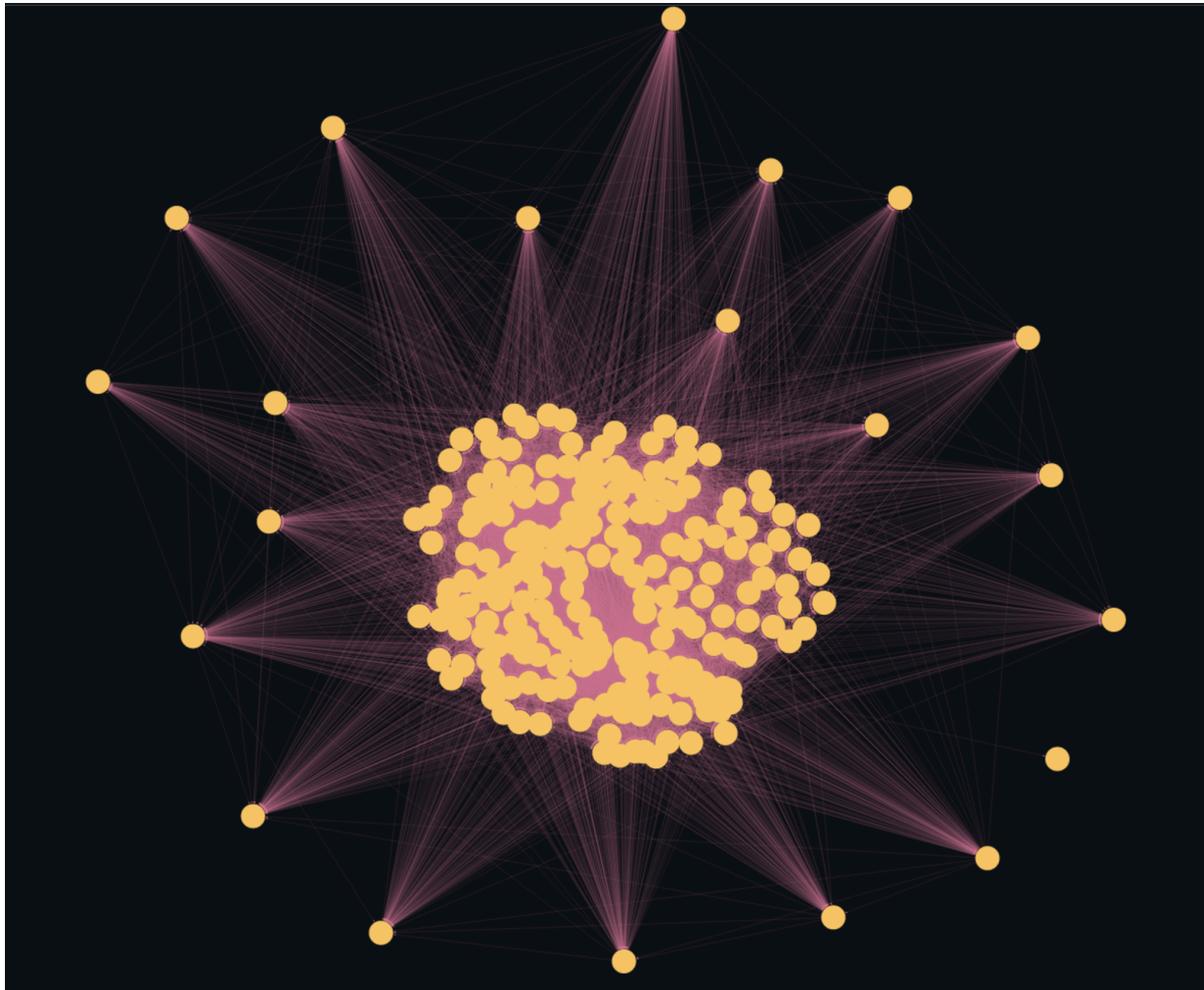


Figure 3.2.3: Lyon School Visualization

3.3 Preprocessing

- **Interaction Frequency Computation:**
 - Data was aggregated by SRC and DEST.
 - The frequency of interactions was calculated and stored in a new column named **FREQ**.
- **Frequency Normalization:**
 - Interaction frequencies were normalized using sci-kit-learn preprocessing tools.
 - Normalized values were saved in a new column, **NormFreq**.
- **Creating a Graph in Neo4j:**
 - Using metadata files, nodes were created for each person with attributes **Pid** and **UserType**.
 - Labeled directed edges with 'follow' were created between SRC and DEST nodes, with normalized frequencies (**NormFreq**) as edge properties.
 - Transactions were batched to allow for efficient processing.

```
first_ds.csv ×
```

```
backend > first_ds.csv
```

	SRC	DEST	FREQ	NormFreq	
1	0	15	17	13	0.0006594785091854292
2	1	15	21	16	0.0008116658574589898
3	2	15	48	16	0.0008116658574589898
4	3	15	50	23	0.001166769670097298
5	4	15	63	19	0.0009638532057325504
6	5	15	66	12	0.0006087493930942424
7	6	15	80	22	0.001116040554006111
8	7	15	102	15	0.0007609367413678029
9	8	15	105	13	0.0006594785091854292
10					

Figure 3.3.1: CSV File of the Dataset (Post-Preprocessing)

3.4 Backend

This project simulates information diffusion within a network using agent-based modeling (ABM). The backend is implemented using Flask, which handles web requests, and Flask-SocketIO for real-time updates between the client and server. The simulation logic utilizes the agentpy library to model agents' behaviors and the networkX library for managing the network graph.

3.4.1 Network Initialization

The system creates a dynamic network graph when the user selects a specific simulation type on the homepage.

- **Nodes (Agents):** Each agent represents an individual in the network, initially marked as uninformed (color-coded blue).
- **Links:** Define relationships between nodes based on the provided adjacency matrix, creating the underlying social network structure.

The adjacency matrix is constructed from a CSV file, where each row defines a source and destination node, with the corresponding relationship strength (**NormFreq**).

This graph is the foundation for simulating information spread, representing social or communication networks in a realistic manner.

3.4.2 Agent-Based Modeling

In our simulation, agents (individuals in the network) transition between states based on their interactions:

- **States:**
 - **Uninformed (Blue):** Default state.
 - **Informed (Red):** Agents who receive and spread information.
- **Information Spread Mechanism:** Informed agents attempt to influence their neighbors with a certain probability. Upon success, neighboring agents become informed, and the status update is broadcast to the front end in real-time.

3.4.3 Simulation Execution

The simulation proceeds through several stages:

- **Setup:** Agents are initialized with a predefined percentage starting as informed. The graph structure is also created at this point.
- **Steps:** During each iteration, informed agents try to influence their neighbors.

- **Termination:** The process halts after a specified number of steps or when all agents are informed, or if no further spread of information occurs.

Metrics such as the **total number of steps** and the **number of informed agents** are recorded throughout the simulation.

3.4.4 Real-Time Updates

The backend leverages **Flask-SocketIO** for real-time data updates. When an agent's state changes (e.g., from uninformed to informed), the backend pushes these updates to the frontend using WebSockets. This ensures that the visualization remains synchronized with the simulation as it progresses.

3.4.5 Code Overview

The backend code for this application is structured as follows:

1. **CSV Parsing:** The `process_csv()` function loads the adjacency matrix from a CSV file, defining the relationships between nodes.
2. **Graph Construction:** The `create_graph_data()` function creates the initial graph structure, where nodes represent agents and edges represent relationships.
3. **Agent Initialization:** The `Person` class models individual agents, and the `InfoDiffusion` class simulates the diffusion of information across the network.
4. **Simulation Execution:** The `run()` method in `InfoDiffusion` runs the simulation, where agents interact, share information, and update their states. The simulation stops when all nodes are informed or when there are no more uninformed agents to influence.
5. **API Endpoints:**
 - `/get_dataset/<datasetId>`: Fetches the network graph data by loading the appropriate CSV dataset.
 - `/simulation/<simulation_id>/step/<int:step>`: Retrieves the state of the simulation at a specific step.
 - `/simulation/<simulation_id>`: Allows cleanup of a simulation instance.
6. **Real-Time Updates with WebSockets:** The Flask-SocketIO integration allows the backend to send updates to the frontend in real-time whenever a node's state changes.

3.4.6 Key Code Sections

```
def process_csv(file_path):
    df = pd.read_csv(file_path)
    nodes = sorted(set(df['SRC']).union(set(df['DEST'])))
    node_index = {node: idx for idx, node in enumerate(nodes)}
    size = len(nodes)
    adj_matrix = np.zeros((size, size))
    for _, row in df.iterrows():
        src_idx = node_index[row['SRC']]
        dest_idx = node_index[row['DEST']]
        adj_matrix[src_idx][dest_idx] = row['NormFreq']
    return adj_matrix, nodes

#####
#Our Simulation Model
#####
class Person:
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self, id):
        self.id = id
        self.condition = 0
        self.color = "blue"

class InfoDiffusion:
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self, params):
        self.steps = 0
        self.G = params['G']
        self.population = params['population']
        self.initial_spread = params['initial_information_spread_share']
        self.simulation_states = []

        # Initialize agents
        self.agents = {}
        for i in range(self.population):
            self.agents[i] = Person(str(i))
            self.agents[i].color = "blue"

        # Set initial informed agents
        initial_informed = int(self.initial_spread * self.population)
        informed_indices = np.random.choice(self.population, initial_informed, replace=False)
        for idx in informed_indices:
            self.agents[idx].condition = 1
            self.agents[idx].color = "red"

        self.simulation_states.append(self.get_current_state())
        self.attempts_without_update = 0
        self.max_attempts_without_update = 100
```

Figure 3.4.1: Backend Code for CSV Data Processing, and ABM Classes

3.4.6 Simulation Control and API

- **Start Simulation:** A simulation is initiated by sending a POST request to `/simulation/<datasetId>`. Parameters such as the initial spread percentage are passed as JSON.
- **Get Simulation State:** The current state of a simulation can be accessed by sending a GET request to `/simulation/<simulation_id>/step/<int:step>`.
- **Stop Simulation:** A DELETE request to `/simulation/<simulation_id>` removes the simulation from the active simulations dictionary, cleaning up resources.

3.4.8 Example Output

For an active simulation, after sending a GET request to `/get_dataset/1`, the returned JSON will look like:

```
{
  "nodes": [
    {"id": "0", "name": "Agent_1", "color": "blue"},
    {"id": "1", "name": "Agent_2", "color": "red"},
    ...
  ],
  "links": [
    {"source": "0", "target": "1"},
    {"source": "2", "target": "3"},
    ...
  ]
}
```

Figure 3.4.2: Simulation Output (Received by the Frontend)

Additionally, once the simulation steps are completed, real-time updates can be pushed to the frontend. For instance, when a node's color changes (e.g., from blue to red), the frontend immediately reflects this change by updating the corresponding node's appearance in the graph, offering live feedback to the user. This continuous stream of updates ensures that the simulation remains synchronized between the backend and the user interface. As a result, our backend architecture facilitates a seamless user experience, with visualizations that stay aligned with the backend model throughout the information diffusion process.



Figure 3.4.3: Simulation Updates of Nodes (Displayed on the Frontend)

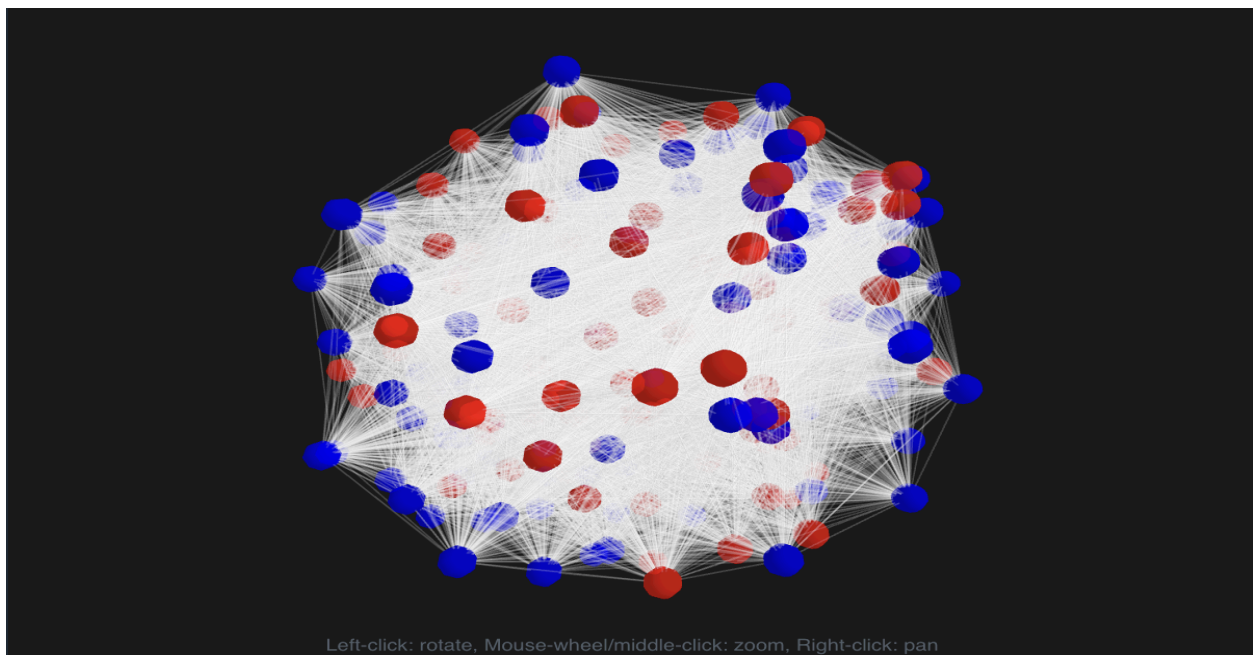


Figure 3.4.4: Realtime Frontend Nodes Updates

3.5 System Requirements

Backend (Python + Flask):

We have used the following libraries for our backend development:

1. **agentpy**: This library is used for building and simulating agent-based models (ABMs). It provides a framework for defining agents, environments, and running simulations, which is crucial for modeling complex systems and interactions within the simulation.
2. **networkx**: This Python library is employed for the creation, manipulation, and analysis of the structure of complex networks. It is particularly useful for representing and working with graphs, which are essential for modeling relationships between agents or entities in our simulation.
3. **numpy**: A fundamental package for scientific computing in Python, numpy is used for handling large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays. It plays a key role in numerical calculations within the backend.
4. **Flask**: A lightweight web framework for Python, Flask is used to build the server-side components of our application. It handles HTTP requests, routes, and serves the backend logic, enabling communication between the frontend and the simulation backend.
5. **flask-cors**: This extension for Flask allows us to handle Cross-Origin Resource Sharing (CORS) issues, enabling our frontend and backend to communicate seamlessly, even when they are hosted on different domains or ports.
6. **flask-socketio**: This library is used to enable real-time, bidirectional communication between the frontend and backend via WebSockets. It is particularly useful for pushing updates to the frontend during the simulation, ensuring the user interface is synchronized with the backend in real-time.
7. **eventlet**: Eventlet is a networking library used to enable asynchronous network applications. In our case, it helps to handle multiple concurrent connections, allowing real-time updates to be pushed from the backend to the frontend without blocking other operations.

Frontend (React + Visualization):

For the frontend, we have used a **React app** with the **Vite bundler**. Here's a breakdown of the core and main libraries that are key to the functionality of our frontend. These libraries are integral to the functionality of the frontend, handling UI rendering, routing, real-time communication with the backend, and styling, all of which contribute to a dynamic and responsive user interface of our frontend.

1. **React** (^18.3.1)

React is the main JavaScript library used for building the user interface (UI). It provides a declarative approach to creating dynamic UIs, enabling us to efficiently update and render components based on changes in the application state.

2. **react-dom** (^18.3.1)

ReactDOM is the package that allows React to interact with the DOM. It provides methods for rendering React components into the HTML DOM and helps in managing the lifecycle of components in a web application.

3. **axios** (^1.7.7)

Axios is a promise-based HTTP client for making requests to the backend. It's used for handling API calls, allowing the frontend to interact with the backend server to fetch data or send requests (like starting a simulation, fetching results, etc.).

4. **react-router-dom** (^7.1.1)

This library is used for handling routing in the React application. It allows us to define different routes (URLs) and associate them with React components, enabling smooth navigation between different pages or views within the app without reloading the page.

5. **socket.io-client** (^4.8.1)

Socket.IO is used for real-time, bidirectional communication between the frontend and backend via WebSockets. The **socket.io-client** package allows the frontend to establish a WebSocket connection with the server, enabling real-time updates, such as simulation progress or node changes, to be pushed to the UI instantly.

CHAPTER 4 : METHODOLOGY

4.1 System Design / Architecture

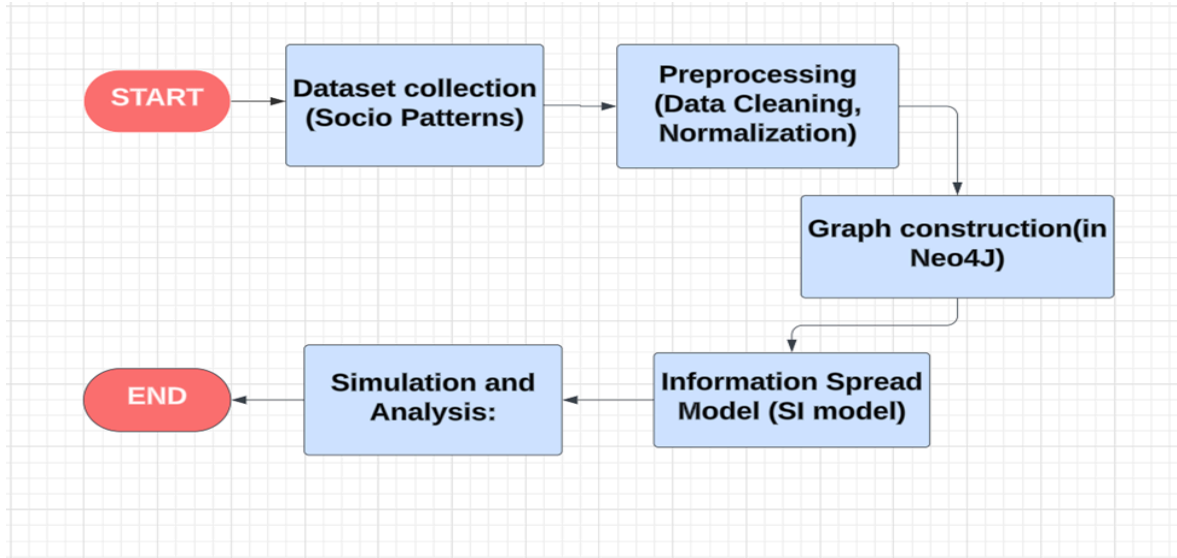


Figure 4.1: System Design

Start (Dataset Collection - Socio Patterns):

The process begins with the collection of datasets, specifically from the "Socio Patterns" project. Socio Patterns provide data about human interactions and social networks.

Preprocessing (Data Cleaning, Normalization):

The raw dataset undergoes preprocessing. This step involves cleaning the data (removing errors, duplicates, or inconsistencies) and normalizing it (ensuring uniformity in scale or format).

Graph Construction (in Neo4j):

After preprocessing, the data is used to construct a graph in Neo4j, a graph database. This graph represents the relationships and interactions within the dataset.

Information Spread Model (SI Model):

The graph is used to apply an information spread model, specifically the SI (Susceptible-Infected) model. This is a common model in epidemiology and social dynamics to simulate how information or diseases propagate through a network.

Simulation and Analysis:

The results of the information spread model are simulated and analyzed to gain insights into the dynamics of information flow or contagion within the network.

4.2 Working Principle:

The SI model is a basic paradigm to simulate an information dissemination process on networks by segregating nodes into three states, showcasing the dynamic process of the state transitions over time. Working principles of the SI model in the context of this project: it generally relies on leveraging graph theory for making representations of social interactions and simulating transitions from one state to another accordingly based on given probabilities. Here's how this will be achieved:

1. State Definitions

SIR models classify each node—that is, each individual in the social network—into one of the following states:

- **Susceptible (S):** Nodes that haven't received the information yet and thus are vulnerable to influence.
- **Infected (I):** Nodes that have received the information and are spreading it to their neighbors.

2. Mechanism of Transition

- **Transmission Probability (β):** Probability that a susceptible node becomes infected when exposed to an infected neighbor.
- At each time step, the model assesses the neighbors of the infected nodes:
- If an infected node interacts with a susceptible neighbor, it may get infected with a probability defined by β

3. Graph Representation and Dynamics

In this project, the network of interactions is represented as a graph, where:

- **Nodes** represent individuals.
- **Edges** depict interactions or communication channels, often weighted to reflect the frequency or strength of connections.

The SI model is iteratively applied across the graph using tools like **Neo4j** for graph storage & queries for agent-based simulations. The simulation proceeds in discrete time steps, updating each node's state based on interactions and probabilistic transitions.

4. Dynamics of Information Spread

The spread of information evolves over time through:

- **Initial Seeding:** A subset of nodes is selected as the initially infected group, marking the information cascade's starting point.
- **Propagation:** Infected nodes attempt to influence susceptible neighbors, with successful transmissions depending on the transmission probability (β).

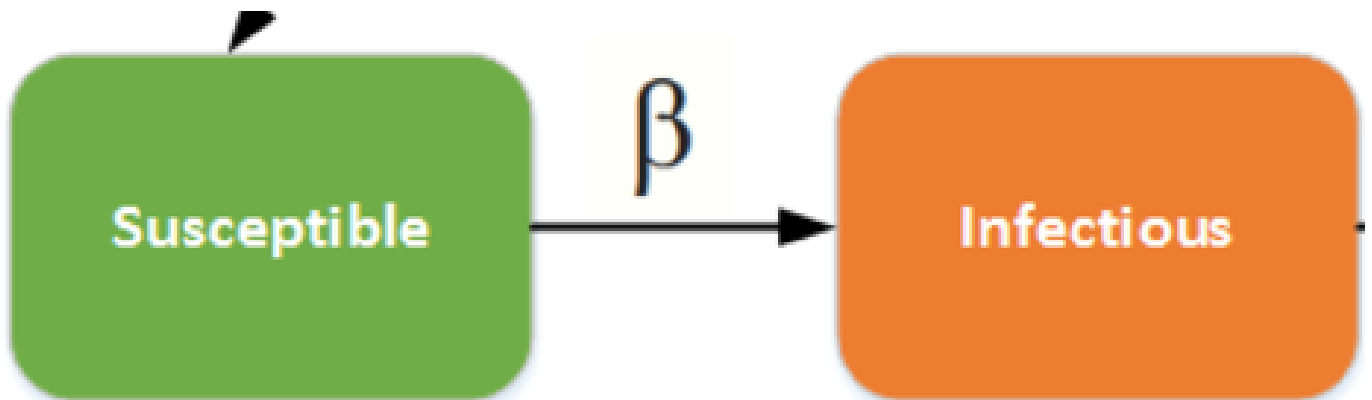


Figure 4.2: Agent State Transition

4.3 Results and Discussions

In this section, we present the results of our network analysis, with a focus on the visualization of social contact networks derived from real-world datasets. These datasets, sourced from repositories like SocioPatterns and SNAP, capture interactions among individuals in various environments such as classrooms, hospitals, and office spaces. Each node in the graph represents an individual, and the edges (connections) between nodes reflect the frequency or intensity of their interactions. The visualizations of these networks are generated using the React Force Graph library, providing an intuitive and dynamic representation of the evolving relationships between individuals.

The React Force Graph component, integrated into the frontend, models the network structure as a directed graph where nodes represent individuals and edges represent interactions. The visualizations enable real-time exploration, with users able to observe how information or behaviors spread across the network as it evolves over time. Densely connected clusters highlight groups of individuals with frequent interactions, while sparse regions represent individuals with fewer connections. This structure allows for the interpretation of patterns such as group formation, information diffusion, and even the potential spread of behaviors or diseases.

To enhance the analysis, we implemented a Flask backend paired with Socket.IO for real-time data streaming. This backend system continuously streams updates about the state of the simulation, ensuring that the frontend application remains synchronized with the latest data. When the simulation is running, the backend emits real-time updates to the frontend, allowing the graph to dynamically change as individuals transition between states, such as becoming "informed" (represented by a red color). This real-time feature enables the simulation of live systems, where users can track the evolution of interactions between individuals.

The simulation itself models information diffusion among individuals in the network. We initialize a certain percentage of individuals as "informed," and at each step of the simulation, informed individuals attempt to pass the information to their uninformed neighbors. This process continues until all individuals are informed or the simulation reaches a stopping condition, such as when no further diffusion occurs.

From a network perspective, the visualizations offer valuable insights. For example, the graph shows clusters of highly connected individuals (informed) and isolated nodes (uninformed). The informed vs uninformed percentages are displayed in the interface, allowing users to track the progress of the information spread. Users can play, pause, or step through the simulation at their own pace, observing how the information spreads and how the network structure evolves over

time. Additionally, the step-based interaction, where users can slide through simulation steps, enables the exploration of different stages of diffusion, providing a deeper understanding of how information propagates across varying network topologies.

The integration of Socket.IO ensures that the frontend application automatically receives updates from the backend in real time, which allows the graph to reflect the dynamic changes as the simulation progresses. This seamless connection between the backend and frontend allows for a truly interactive and responsive analysis experience, where users can explore the network's evolution, the spread of information, and potentially simulate scenarios in which behaviors or diseases spread through populations.

In summary, the interactive and real-time features of the simulation—enabled by React, Flask, and Socket.IO—offer a powerful framework for visualizing and analyzing dynamic social networks. By leveraging these technologies, we gain valuable insights into network structures, the diffusion of information, and other complex systems, providing useful tools for understanding real-world phenomena such as social influence, group dynamics, and epidemic spread.

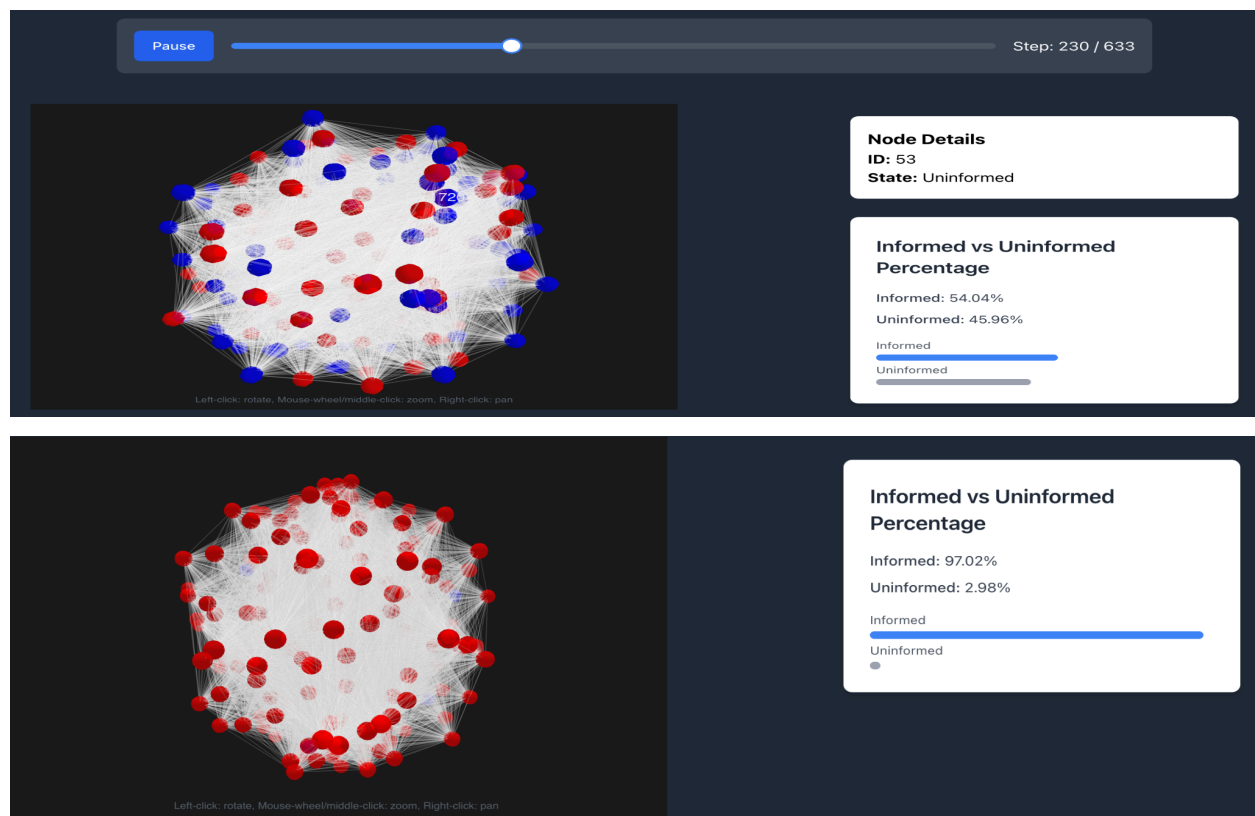


Figure 4.3.1: Real-time 3D-Graph Visualization

CHAPTER 5. CONCLUSION

In conclusion, our project successfully modeled the diffusion of information within real-world social networks, using the SI (Susceptible-Infected) model to simulate how information spreads among connected individuals. By leveraging React Force 3D for graph-based visualization and NetworkX for network analysis in Python, we were able to create a dynamic and accurate representation of information propagation in social systems. The project was built on real-world interaction data sourced from the SocioPatterns and SNAP repositories, making our simulations both realistic and reflective of actual human behavior in diverse environments such as classrooms, hospitals, and office spaces.

The results of our project underscore the importance of understanding the dynamics of information spread in social networks. Through our analysis, we observed how individuals' behaviors and interactions evolved over time, with key nodes (representing individuals) transitioning between "susceptible" and "infected" states. These findings are critical for understanding the speed and reach of information diffusion in practical contexts like digital marketing, public health communication, and viral phenomena.

The real-time nature of our system, built with Flask and Socket.IO, enabled us to observe how information spread dynamically, reflecting the continuous changes in the network structure as new data was introduced. This real-time capability provides valuable insights for real-world applications, such as managing the spread of misinformation or planning public health campaigns. The React Force 3D visualizations also offered an intuitive way to explore the changing relationships within the network, allowing us to track how individual nodes interacted over time and how the overall structure evolved.

Looking forward, our project lays the groundwork for several potential improvements. One key direction is the integration of more advanced models, such as the SIR (Susceptible-Infected-Recovered) and SIS (Susceptible-Infected-Susceptible) models, which better account for real-world dynamics where individuals may lose interest or be re-exposed to information. We also plan to extend our approach to agent-based modeling, where each node (representing an individual) can make independent decisions based on its interactions, thus providing a more granular simulation of human behavior.

In terms of future enhancements, we aim to unify and integrate multiple datasets from SocioPatterns, SNAP, and others into a cohesive system, enhancing the robustness and versatility of our model. We will also expand the visualization options available in React Force Graph, exploring additional layouts like hierarchical and force-directed graphs to better represent the

diverse structures of social networks. Moreover, we plan to improve the user interface, making it even more interactive and accessible for users to explore and analyze the network dynamics.

By enhancing the scalability and real-time capabilities of the system, we will be able to study larger and more complex networks, providing valuable insights for sectors such as social media analytics, public policy, and consumer behavior analysis. Understanding how information spreads across networks is crucial for decision-making and strategic planning in these fields.

In summary, this project demonstrates the power of combining network analysis with interactive, real-time visualization tools like React Force 3D. It highlights the importance of studying information diffusion in social networks and sets the stage for future developments that will make these tools even more powerful and applicable in real-world scenarios.

CHAPTER 6. REFERENCE AND PUBLICATIONS

Wikipedia. (n.d.). Compartmental Models in Epidemiology.

Link: https://en.wikipedia.org/wiki/Compartmental_models_in_epidemiology

Provides an overview of basic epidemiological models like SIR, SIS, and SEIR, which serve as the foundation for the project's simulations.

ArXiv. (n.d.). Influence Maximization in Social Networks.

Link: <https://arxiv.org/abs/>

This paper offers detailed explanations of algorithms for influence propagation in networks, relevant to identifying key influencers for information diffusion.

EON: Exponential Random Graph Models.

Link: <https://link.springer.com/book/10.1007/978-3-319-50806-1>

This book offers foundational knowledge on disease spread modeling and highlights the limitations of traditional approaches, relevant to improving our modeling framework.

Barabási, A.-L. (2016). Network Science. Cambridge University Press.

Link: <https://networksciencebook.com/>

A comprehensive source of knowledge on network science, covering models and simulations, including popular differential equation-based approaches used in network Modeling.

Wikipedia. (n.d.). Chaos theory

Link: https://en.wikipedia.org/wiki/Chaos_theory

The Wikipedia article on **Chaos Theory** provides an overview of the key concepts, historical development, mathematical foundations, and applications of chaos theory. . It explains the phenomenon of deterministic systems that can produce highly unpredictable and complex behavior, even though they are governed by specific, deterministic laws.

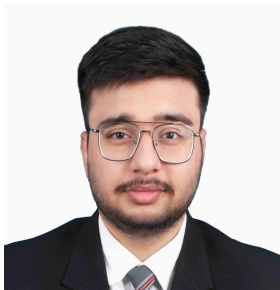
Github - React Force Graph Library (Open Source)

Link: <https://github.com/vasturiano/react-force-graph>

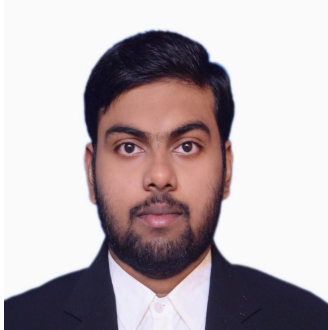
The **React Force Library** is a powerful tool for creating interactive, force-directed graph visualizations in React applications. It leverages the D3.js force simulation to enable dynamic and visually engaging representations of network structures, making it ideal for exploring complex data relationships.

CHAPTER 7. TEAM MEMBERS BIODATA

1. Braj Kishor Sharma 21BCE11078

	Registration Number: 21BCE11078 Email: braj.kishor2021@vitbhopal.ac.in Phone: +91 7987728652 LinkedIn: https://www.linkedin.com/in/braj-sharma/ GitHub: https://github.com/Braj111 Contributions: Frontend Modelling, Simulation & UIUX Backend Integration
Technical Skills	Java, Python, HTML, CSS, JavaScript, Typescript, Bootstrap, Tailwind, React, Node, Express, SQL, NoSQL
Certification	The Bits and Bytes of Computer Networking, Coursera. AWS Certified Cloud Practitioner Oracle Java SE11

2. Arijit Modak 21BCE11159

	Registration Number: 21BCE11159 Email: arijit.modak2021@vitbhopal.ac.in Phone: +91 6295401465 LinkedIn: https://www.linkedin.com/in/arijit-modak-b42729221/ GitHub: https://github.com/Arijit2003 Contributions: Backend Core API Development, Project Documentation
Technical Skills	Java, Python, Javascript, Typescript HTML, CSS, React(MERN), Android Developer
Certification	The Bits and Bytes of Computer Networking, Coursera. AWS Certified Cloud Practitioner Oracle Java SE11

3. Devashish Dhaulakhandi 21BCE10076

	Registration Number: 21BCE10076 Email: devashish.dhaulakhandi2021@vitbhopal.ac.in Phone: +91 9953951873 GitHub: https://github.com/Devashish-Dh Contributions: Backend Development (Agent Based Modelling) Research - Information Diffusion Modelling
Technical Skills	The Bits and Bytes of Computer Networking, Coursera. Python, C, C++

4. Aswin P Kumar 21BCE11072

	Registration Number: 21BCE11072 Email: aswin.p2021@vitbhopal.ac.in Phone: +91 7356898098 LinkedIn : GitHub : Contributions: Dataset Collection , Study & Cleaning Backend Development & Synchronizations
Technical Skills	Python, Java, SQL, Selenium
Certification	The Bits and Bytes of Computer Networking, Coursera. AWS Certified Cloud Practitioner

5. Akshant Ujjwal 21BCE11650

	Registration Number: 21BCE11650 Email: akshant.ujjwal2021@vitbhupal.ac.in Phone: +91 6395883886 LinkedIn: https://www.linkedin.com/in/akshant-ujjwal-47a533273/ GitHub: https://github.com/AkshantUjjwal Contributions: Project Research Work Project Documentations
Technical Skills Certification	Java, C++, Python Applied Machine Learning in Python, Coursera, Cloud Digital Leader, Google.